RTL Design of a KCF Visual Tracker

Spring 2026

Submitted by

Akhil Sriram (Roll No. [2210110134.])

Under Supervision of

Dr. Venkatnarayan Hariharan
Department of Electrical Engineering

Department of Electrical Engineering
School of Engineering
Shiv Nadar University

(Spring 2026)

Abstract

Visual object tracking is a critical task in computer vision, but traditional software implementations on general-purpose processors often suffer from high latency and power consumption. This project presents a purely hardware-based implementation of the MOSSE (Minimum Output Sum of Squared Error) correlation filter tracking algorithm, designed entirely in Verilog HDL. Unlike software solutions that run sequentially on a CPU, this project leverages the parallel processing capabilities of an FPGA (Field-Programmable Gate Array) to perform high-speed tracking. The design focuses on translating the mathematical operations of the MOSSE algorithm—specifically the Fast Fourier Transforms (FFT) and complex conjugate multiplications—into dedicated digital logic circuits (RTL). The system is designed to operate as a standalone IP core. It accepts image data (such as pixel streams or memory initialization files) as input, processes the frames using a pipeline of custom hardware modules, and outputs the target coordinates. The implementation will be verified through behavioral simulation and synthesized on an FPGA to demonstrate the efficiency and speed advantages of a dedicated hardware solution compared to software-based execution.

Contents

1 Introduction **7**
1.1 Background and Motivation . . . 7
1.2 The KCF Advantage . . . 7
1.3 Scope of the Mid-Semester Presentation . . . 7

2 Literature Survey **9**
2.1 The MOSSE Filter . . . 9
2.2 The KCF Tracker . . . 9
2.3 MOSSE vs. KCF: Comparison . . . 10
2.4 Justification for Choosing KCF . . . 10
2.5 Hardware Acceleration of the FFT . . . 11

3 KCF Theory **12**
3.1 Circulant Structure and the DFT . . . 12
3.2 Training: Learning the Filter . . . 12
3.3 Detection: Locating the Target . . . 13
3.4 Combined Filter for Hardware . . . 13
3.5 DFT Displacement Convention . . . 14

4 Work Done **15**
4.1 Algorithm Pipeline Overview . . . 15
4.2 Detection Pipeline . . . 15
4.3 System Architecture and FSM . . . 16
4.4 Key Building Blocks . . . 17
4.4.1 The Butterfly Unit . . . 17
4.4.2 Twiddle Factors . . . 17
4.4.3 1D FFT (32-Point) . . . 17
4.4.4 2D FFT (Row-Column Decomposition) . . . 18
4.4.5 2D IFFT (Conjugate Trick) . . . 18
4.5 Fixed-Point Arithmetic . . . 18
4.5.1 The Scaling Challenge . . . 18

5 Results and Verification **20**
5.1 Simulation Environment . . . 20
5.2 Synthetic Test Verification . . . 20
5.3 Vivado Waveform Analysis . . . 20
5.4 Real-Image Demonstration . . . 21

5.4.1 Target Selection and Training 21
5.4.2 Detection on Shifted Frame 23
5.4.3 Response Map Analysis 23
5.4.4 End-to-End Pipeline Visualisation 24
5.4.5 Hardware–Software Agreement 25

6 Future Work **27**
6.1 Next Objectives 27
References 28

List of Figures

3.1 Conceptual illustration of KCF frame-to-frame tracking. In Frame 1, the target is at position $(2, 3)$. In Frame 2, the target has moved one pixel to the right to $(2, 4)$. The KCF detection response peaks at the displacement $(0, +1)$, correctly identifying the motion. 13

4.1 High-level overview of the KCF algorithm showing both the training and tracking stages. The training stage learns the filter model from an initial frame, while the tracking stage uses it to locate the target in subsequent frames. 15

4.2 Hardware detection pipeline block diagram. Data flows left to right through seven stages controlled by a central FSM. The ROM now stores the combined filter $\overline{\hat{\alpha}} \odot \hat{X}$ rather than $\hat{\alpha}$ alone, enabling correct cross-frame detection with a single conjugate multiply. 16

5.1 Full Vivado simulation waveform showing the complete detection pipeline execution. The patch is loaded during the first $\sim 5\,\mu$s, followed by windowing, FFT, multiplication, IFFT, and peak finding stages. 21

5.2 Zoomed-in view of the simulation output at $t \approx 193,465\,$ns. The `done` pulse (highlighted) coincides with the outputs settling to the correct peak coordinates, matching the golden reference. 22

5.3 Frame 1 — the input image with the 32×32 target region highlighted (red box). The target is centred at pixel (240, 300), chosen for its rich ground texture. 22

5.4 Frame 2 — the search region (blue box) is shifted by $(+3, +2)$ pixels from the target centre. The red star marks the detected target position, correctly displaced from the search centre by $(-3, -2)$ pixels. 23

5.5 Centred detection response map. The axes show displacement in pixels from the expected target position. The peak (cyan star) at $(-3, -2)$ correctly matches the simulated target motion of 3 pixels down and 2 pixels right. 24

5.6 End-to-end KCF detection pipeline on a real image. (a) Target patch from Frame 1. (b) Hann-windowed target — edges taper to zero to prevent spectral leakage. (c) Search patch from Frame 2, shifted by $(+3, +2)$ pixels. (d) Centred response map showing the detection peak at displacement $(-3, -2)$, correctly identifying the simulated motion. 25

5.7 Terminal output from the real-image demo. The Python script trains the filter, generates `.mem` files, and reports the golden detection peak at $(29, 30)$ with displacement $(-3, -2)$. The Verilog simulation (not shown separately) produces the identical peak, confirming hardware–software agreement on real image data. 26

Chapter 1

Introduction

1.1 Background and Motivation

Real-time visual tracking is a fundamental component in robotics, autonomous navigation, and smart surveillance systems. The primary challenge in these domains is tracking a fast-moving Region of Interest (ROI) with minimal latency and low power consumption. Software-based implementations often struggle to meet these strict timing constraints without relying on power-hungry processors.

By translating the tracking algorithm into a custom hardware accelerator (IP core), we can achieve cycle-accurate, highly parallelized execution, drastically reducing the time required to process each incoming video frame.

1.2 The KCF Advantage

Traditional spatial-domain tracking algorithms rely on sliding a template across a search window and computing metrics like Sum of Absolute Differences (SAD) or Normalized Cross-Correlation (NCC). This approach is highly inefficient, as the time complexity scales quadratically with the image size.

The Kernelized Correlation Filter (KCF) [1] elegantly sidesteps this bottleneck. By leveraging the properties of circulant matrices, KCF transforms the image patch into the frequency domain using a 2D Fast Fourier Transform (2D-FFT). In the frequency domain, massive spatial convolutions are reduced to simple element-wise multiplications, allowing for immense computational speedups when mapped to hardware.

1.3 Scope of the Mid-Semester Presentation

To demonstrate the viability of the hardware approach within the mid-semester timeline, the following simplifications were adopted:

- **32×32 patch size** instead of the full 64×64, reducing memory and FFT complexity by 4×.
- **Linear kernel** instead of a Gaussian kernel, eliminating the need for a hardware exponential function.

- **Detection only**: the model is trained offline in Python and the frozen filter coefficients are loaded from ROM. The online model update loop is deferred to the next phase.
- **No AXI bus interface**: pixels are loaded directly via a simple write port for simulation.

Chapter 2

Literature Survey

2.1 The MOSSE Filter

The Minimum Output Sum of Squared Error (MOSSE) filter, introduced by Bolme et al. [2], was among the first trackers to demonstrate that frequency-domain correlation could achieve real-time speeds while maintaining competitive accuracy. The core idea is to learn a filter H^* in the frequency domain that, when correlated with an image patch F, produces an output G that peaks sharply at the target location:

$$G = H^* \odot F \tag{2.1}$$

The optimal filter is obtained by minimising the sum of squared errors between the actual correlation output and a desired Gaussian-shaped response G_i over a set of training frames F_i:

$$H^* = \frac{\sum_i G_i \odot \overline{F_i}}{\sum_i F_i \odot \overline{F_i}} \tag{2.2}$$

where $\overline{(\cdot)}$ denotes complex conjugation. Because all operations are element-wise in the frequency domain, MOSSE runs at several hundred frames per second on a single CPU core. It also supports online model updates by maintaining running averages of the numerator and denominator, allowing the filter to adapt to appearance changes over time.

However, MOSSE has a fundamental limitation: it operates on **raw pixel intensities** and learns a strictly **linear** relationship between the input patch and the desired response. This means the filter can only capture linear correlations in the image, limiting its ability to distinguish the target from visually similar distractors or to handle non-linear appearance variations such as partial occlusions, illumination changes, and in-plane rotations.

2.2 The KCF Tracker

The Kernelized Correlation Filter (KCF), proposed by Henriques et al. [1], builds directly on the MOSSE framework but introduces a critical enhancement: the **kernel trick**. Instead of correlating raw pixel patches linearly, KCF first maps the patches into a higher-dimensional feature space using a kernel function $\kappa(\mathbf{x}, \mathbf{x}')$:

$$k^{xx'}[m] = \kappa(\mathbf{P}^m \mathbf{x},\ \mathbf{x}') \tag{2.3}$$

where $\mathbf{P}^m\mathbf{x}$ is the m-th cyclic shift of $\mathbf{x}$. The key theoretical insight of KCF is that kernel correlations of circulant data can still be diagonalised by the DFT [1], preserving the $O(N^2 \log N)$ computational advantage while operating in the richer kernel feature space.

Common kernel choices include:

- **Linear kernel:** $\kappa(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T\mathbf{x}'$ — equivalent to MOSSE, but derived within the KCF framework.
- **Gaussian (RBF) kernel:** $\kappa(\mathbf{x}, \mathbf{x}') = \exp(-\|\mathbf{x} - \mathbf{x}'\|^2/2\sigma^2)$ — captures non-linear relationships, significantly improving robustness.

2.3 MOSSE vs. KCF: Comparison

Table 2.1 summarises the key differences between the two trackers. While both share the same FFT-based computational backbone, KCF consistently outperforms MOSSE in tracking accuracy due to the kernel extension.

Table 2.1: Comparison of MOSSE and KCF trackers.

Property	**MOSSE**	**KCF**
Feature space	Raw pixels (linear)	Kernel-mapped (non-linear)
Core operation	$H^* \odot F$	$\hat{\alpha} \odot \hat{k}^{xz}$
Computational backbone	FFT / IFFT	FFT / IFFT
Training complexity	$O(N^2 \log N)$	$O(N^2 \log N)$
Detection complexity	$O(N^2 \log N)$	$O(N^2 \log N)$
Robustness to clutter	Low	High
Handles non-linear appearance changes	No	Yes (with Gaussian kernel)
OTB-2013 avg. precision [1]	0.669	0.732
Hardware complexity	Low	Low (linear) / Moderate (Gaussian)

2.4 Justification for Choosing KCF

Both MOSSE and KCF share the same computational backbone — an FFT, an element-wise multiply, and an IFFT — which means the hardware pipeline is nearly identical in both cases. The only difference is what gets stored in the filter ROM and how the element-wise multiplication is configured. Given that the hardware cost is essentially the same, the choice came down to tracking accuracy and extensibility.

KCF was selected for the following reasons:

1. **Strictly superior accuracy at equal cost:** With a linear kernel, KCF reduces to MOSSE but is derived within a more principled ridge-regression framework, giving slightly better regularisation behaviour. The hardware pipeline is identical.
2. **Upgrade path to Gaussian kernel:** The KCF framework naturally accommodates a Gaussian kernel upgrade. This would require adding a single hardware

exponential LUT to the pipeline but would yield a significant jump in tracking robustness — a path that is not available within the MOSSE formulation.

3. **Broader research ecosystem:** KCF has spawned numerous extensions (DCF, CN, DSST [3]) that build on the same circulant structure. Choosing KCF as the base architecture ensures compatibility with future algorithmic improvements.

4. **Published benchmarks:** KCF achieves 73.2% average precision on OTB-2013 compared to MOSSE's 66.9% [1], a 9.4% relative improvement, while maintaining the same frame rate.

In summary, since the hardware cost of implementing KCF is no greater than MOSSE for the linear-kernel case, and KCF provides both better accuracy and a clear upgrade path, it was the natural choice for this project.

2.5 Hardware Acceleration of the FFT

The critical path in any KCF hardware implementation is the Fourier Transform. Efficient RTL designs utilize the Cooley-Tukey algorithm [4], specifically the Radix-2 Decimation-In-Time (DIT) architecture. By iteratively passing data through Butterfly units, the mathematical complexity is reduced from $O(N^2)$ to $O(N \log N)$, making it viable for synthesizable logic.

Chapter 3

KCF Theory

This chapter presents the mathematical foundation of the Kernelized Correlation Filter as it applies to the hardware implementation. The derivation follows Henriques et al. [1], specialised to the linear-kernel case used in this MVP.

3.1 Circulant Structure and the DFT

KCF exploits the fact that all cyclic shifts of a base image patch $\mathbf{x}$ form a *circulant matrix* $C(\mathbf{x})$. A key property of circulant matrices is that they are diagonalised by the Discrete Fourier Transform (DFT):

$$C(\mathbf{x}) = F^{-1} \operatorname{diag}(\hat{\mathbf{x}})\, F \tag{3.1}$$

where F is the DFT matrix and $\hat{\mathbf{x}} = \mathcal{F}\{\mathbf{x}\}$ is the DFT of the base sample. This means that any operation involving all cyclic shifts of $\mathbf{x}$ — such as training a classifier on them — can be computed entirely in the frequency domain with element-wise operations, reducing the complexity from $O(N^4)$ in the spatial domain to $O(N^2 \log N)$ using the FFT.

3.2 Training: Learning the Filter

During training, the tracker is given an image patch $\mathbf{x}$ (the target) and a desired regression label $\mathbf{y}$, which is a 2D Gaussian centred at the origin. The goal is to learn filter coefficients $\boldsymbol{\alpha}$ such that applying the filter to $\mathbf{x}$ (and all its cyclic shifts) produces $\mathbf{y}$.

For the **linear kernel**, the kernel correlation between $\mathbf{x}$ and itself in the frequency domain is simply:

$$\hat{k}^{xx} = \overline{\hat{\mathbf{x}}} \odot \hat{\mathbf{x}} = |\hat{\mathbf{x}}|^2 \tag{3.2}$$

where $\overline{(\cdot)}$ denotes complex conjugation and $\odot$ denotes element-wise (Hadamard) multiplication. The filter coefficients are then obtained by ridge regression in the frequency domain:

$$\hat{\boldsymbol{\alpha}} = \frac{\hat{\mathbf{y}}}{\hat{k}^{xx} + \lambda} = \frac{\hat{\mathbf{y}}}{|\hat{\mathbf{x}}|^2 + \lambda} \tag{3.3}$$

where λ is a regularisation parameter (set to 10^{-2} in this implementation) that prevents

division by zero and improves numerical stability. The Gaussian label $\mathbf{y}$ is defined as:

$$y[r, c] = \exp\left(-\frac{r_{\text{circ}}^2 + c_{\text{circ}}^2}{2\sigma^2}\right) \tag{3.4}$$

where $r_{\text{circ}} = \min(r,\, N - r)$ and $c_{\text{circ}} = \min(c,\, N - c)$ are circular distances and $\sigma = 2.0$ controls the sharpness of the desired response peak.

3.3 Detection: Locating the Target

Given a new search patch $\mathbf{z}$ from the next video frame, the detection response is computed as:

$$\text{response} = \mathcal{F}^{-1}\left\{\overline{\hat{\mathbf{x}}} \odot \hat{\mathbf{z}} \odot \hat{\boldsymbol{\alpha}}\right\} \tag{3.5}$$

The peak of this response map gives the displacement $(\Delta r, \Delta c)$ of the target between frames. Intuitively, the operation computes the correlation between the search patch and the trained template across all possible cyclic shifts simultaneously — all within a single forward FFT, one element-wise multiply, and one inverse FFT.

3.4 Combined Filter for Hardware

Equation (3.5) involves three frequency-domain arrays ($\hat{\mathbf{x}}$, $\hat{\boldsymbol{\alpha}}$, $\hat{\mathbf{z}}$), but the hardware multiplier computes only $a \cdot \overline{b}$ (conjugate multiplication). To reduce the detection to a single conjugate multiply, we **pre-combine** the training data into a single stored filter:

$$\hat{\mathbf{f}}_{\text{stored}} = \overline{\hat{\boldsymbol{\alpha}}} \odot \hat{\mathbf{x}} \tag{3.6}$$

At detection time, the hardware computes:

$$\hat{\mathbf{z}} \cdot \overline{\hat{\mathbf{f}}_{\text{stored}}} = \hat{\mathbf{z}} \cdot \hat{\boldsymbol{\alpha}} \cdot \overline{\hat{\mathbf{x}}} = \overline{\hat{\mathbf{x}}} \cdot \hat{\mathbf{z}} \cdot \hat{\boldsymbol{\alpha}} \tag{3.7}$$

which is exactly Equation (3.5). This formulation has two advantages: (1) only one ROM is needed instead of two, and (2) the conjugate multiplier module (`cconj_mul.sv`) already exists in the pipeline and requires no modification.

Figure 3.1 illustrates the conceptual tracking flow from one frame to the next.

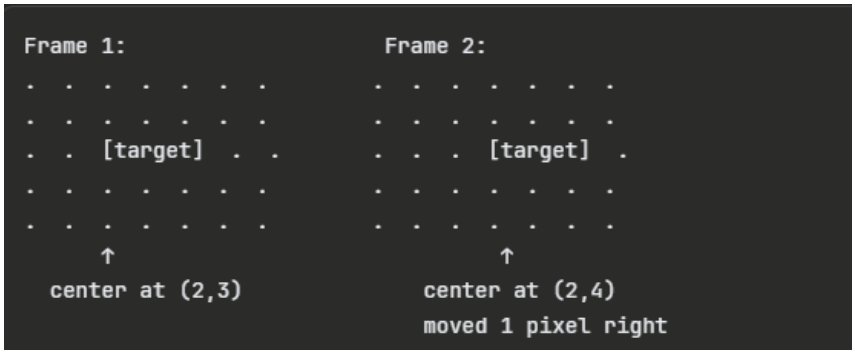

Figure 3.1: Conceptual illustration of KCF frame-to-frame tracking. In Frame 1, the target is at position $(2, 3)$. In Frame 2, the target has moved one pixel to the right to $(2, 4)$. The KCF detection response peaks at the displacement $(0, +1)$, correctly identifying the motion.

3.5 DFT Displacement Convention

The response map produced by the IFFT uses DFT circular indexing. The displacement corresponding to a peak at index (r, c) in an $N \times N$ response map is:

$$\Delta r = \begin{cases} r & \text{if } r < N/2 \\ r - N & \text{if } r \geq N/2 \end{cases}, \qquad \Delta c = \begin{cases} c & \text{if } c < N/2 \\ c - N & \text{if } c \geq N/2 \end{cases} \tag{3.8}$$

For example, with $N = 32$, a displacement of $(\Delta r, \Delta c) = (-3, -2)$ appears as a peak at index $(29, 30)$ in the raw response map. When the response map is centred using an `fftshift` operation, the peak moves to the visually intuitive location near the centre.

Chapter 4

Work Done

4.1 Algorithm Pipeline Overview

The KCF algorithm consists of two primary stages: the **Training Stage** (updating the model) and the **Tracking Stage** (detecting the target). Figure 4.1 illustrates this high-level workflow.

For the mid-semester MVP, the focus is entirely on accelerating the **Tracking Stage**. To isolate and verify the detection math, the model update phase is bypassed by training the model offline in Python and loading the frozen combined filter coefficients from a `.mem` file.

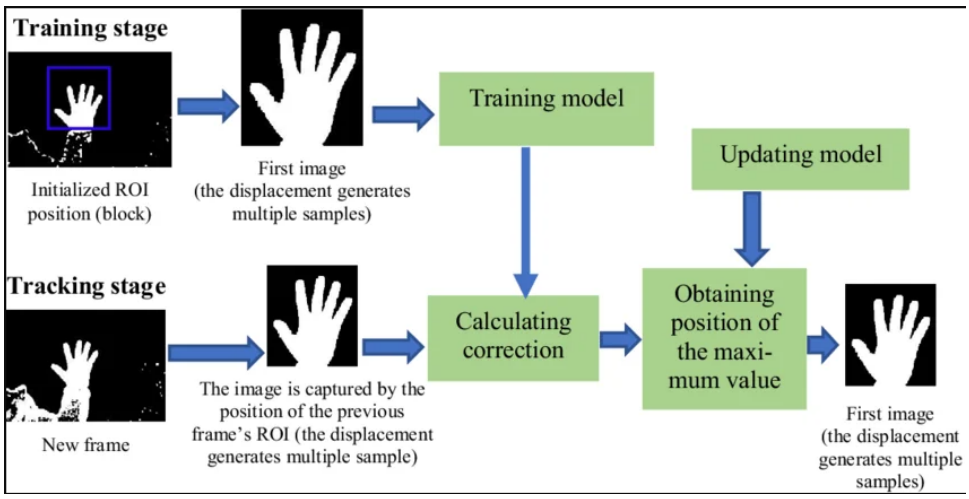

Figure 4.1: High-level overview of the KCF algorithm showing both the training and tracking stages. The training stage learns the filter model from an initial frame, while the tracking stage uses it to locate the target in subsequent frames.

4.2 Detection Pipeline

The core of this project is the hardware detection pipeline. Given a new search patch $\mathbf{z}$ and the pre-computed combined filter $\hat{\mathbf{f}}_{\text{stored}} = \overline{\hat{\boldsymbol{\alpha}}} \odot \hat{\mathbf{x}}$ (see Section 3.4), the pipeline

computes:

$$\text{response} = \text{IFFT}\left(\hat{\mathbf{z}} \cdot \overline{\hat{\mathbf{f}}_{\text{stored}}}\right) = \text{IFFT}\left(\overline{\hat{\mathbf{x}}} \odot \hat{\mathbf{z}} \odot \hat{\boldsymbol{\alpha}}\right) \tag{4.1}$$

where the combined filter is stored in ROM and the conjugate multiplication is performed by the `cconj_mul` module. The peak of the response map gives the (row, col) displacement of the target from its expected position.

Figure 4.2 shows the block-level view of the hardware detection pipeline.

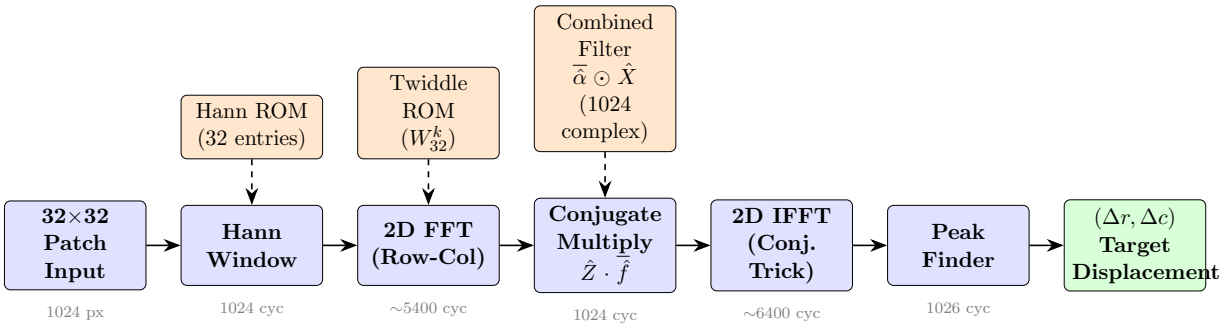

Figure 4.2: Hardware detection pipeline block diagram. Data flows left to right through seven stages controlled by a central FSM. The ROM now stores the combined filter $\overline{\hat{\alpha}} \odot \hat{X}$ rather than $\hat{\alpha}$ alone, enabling correct cross-frame detection with a single conjugate multiply.

4.3 System Architecture and FSM

The top-level module, `kcf_detect_top.sv`, orchestrates a 7-stage Finite State Machine (FSM) that processes the patch sequentially:

1. **S_HANN_WIN** (1024 cycles): Applies a 2D Hann window to the incoming 32×32 image patch. The Hann window smoothly tapers the edges of the patch to zero, preventing sharp discontinuities that would create misleading high-frequency artifacts in the FFT output. The 2D window is computed as the outer product of two 1D Hann vectors, with the coefficients stored in ROM.

2. **S_FFT_LOAD** (1024 cycles): Loads the windowed patch into the 2D FFT module's internal buffer, one pixel per clock cycle.

3. **S_FFT_RUN** (∼5400 cycles): Computes the 2D FFT to bring the spatial-domain image data into the frequency domain. Internally, this processes 32 rows and then 32 columns, each requiring ∼83 clock cycles.

4. **S_ELEM_MUL** (1024 cycles): Performs element-wise conjugate multiplication between $\hat{Z}$ (the FFT of the search patch) and the stored combined filter $\hat{\mathbf{f}}_{\text{stored}}$. This single operation replaces the expensive spatial convolution.

5. **S_IFFT_LOAD** (1024 cycles): Loads the resulting frequency-domain product into the 2D IFFT module.

6. **S_IFFT_RUN** (∼6400 cycles): Computes the 2D Inverse FFT to transform the product back into the spatial domain, producing the response map.

7. **S_PEAK_RUN** (1026 cycles): Scans the entire response map sequentially to find the pixel with the maximum value. The row and column of this peak are the final tracking output.

The total detection latency is approximately **19,300 clock cycles**. At a 100 MHz system clock, this corresponds to ∼193 μs per frame.

4.4 Key Building Blocks

4.4.1 The Butterfly Unit

The butterfly is the atomic computation of the FFT. Named for the shape of its signal-flow diagram, it takes two complex inputs (a and b) along with a pre-computed rotation factor called a *twiddle factor* (W), and produces two outputs:

$$\text{out}_a = a + W \cdot b \tag{4.2}$$

$$\text{out}_b = a - W \cdot b \tag{4.3}$$

This operation is entirely combinational in our design and completes in a single clock cycle. To prevent numerical overflow during successive additions, a *scaled* variant divides both outputs by 2 (implemented as a single-bit arithmetic right shift `>>> 1`).

4.4.2 Twiddle Factors

Twiddle factors are the "rotation angles" that the FFT uses to decompose a signal into its frequency components. For a 32-point FFT, they are defined as:

$$W_{32}^k = e^{-j \cdot 2\pi k/32} = \cos\left(\frac{2\pi k}{32}\right) - j \sin\left(\frac{2\pi k}{32}\right), \quad k = 0, 1, \ldots, 15 \tag{4.4}$$

These 16 complex values are pre-computed in Python and stored in a ROM (`twiddle_rom_32.sv`), with real and imaginary parts interleaved. The symmetry property $W^{k+N/2} = -W^k$ means only half the values need to be stored.

4.4.3 1D FFT (32-Point)

The 32-point 1D FFT (`fft1d_32.sv`) uses an iterative single-butterfly architecture based on the Cooley-Tukey Radix-2 Decimation-In-Time (DIT) algorithm. The key steps are:

1. **Bit-Reversal Permutation:** The 32 input samples are reordered by reversing their 5-bit binary indices (e.g., index 3 = `00011` maps to index 24 = `11000`). This reordering allows the subsequent butterfly stages to operate on adjacent memory locations.

2. **5 Butterfly Stages:** Each stage performs 16 butterfly operations. The "distance" between butterfly pairs doubles at each stage (1, 2, 4, 8, 16), progressively combining more of the input data.

3. **Total:** $5 \times 16 = 80$ butterfly operations + overhead $\approx$ 83 clock cycles per 1D transform.

4.4.4 2D FFT (Row-Column Decomposition)

The 2D FFT (`fft2d_32.sv`) decomposes the 2D transform into two passes of 1D transforms:

1. **Row Pass:** Apply the 1D FFT to each of the 32 rows (with scaling enabled to prevent overflow).

2. **Column Pass:** Apply the 1D FFT to each of the 32 columns of the row-transformed result (without scaling, to preserve signal magnitude).

This asymmetric scaling strategy (rows scaled, columns unscaled) was critical to solving the fixed-point precision challenge, as discussed in Section 4.5.

4.4.5 2D IFFT (Conjugate Trick)

Computing an inverse FFT from scratch would require a separate hardware module. Instead, the IFFT is implemented by reusing the same FFT hardware through the *conjugate trick*:

$$\mathrm{IFFT}(X) = \overline{\mathrm{FFT}(\overline{X})} \ / \ N^2 \tag{4.5}$$

The steps are: (1) negate all imaginary parts of the input (conjugate), (2) run the forward FFT, (3) negate all imaginary parts of the output (conjugate again). The $1/N^2$ normalization is handled automatically by the scaled butterfly architecture.

4.5 Fixed-Point Arithmetic

All computations use Q8.8 fixed-point representation: 16-bit signed values with 8 integer bits and 8 fractional bits. This format can represent values from -128 to $+127.996$ with a resolution of $1/256 \approx 0.004$.

4.5.1 The Scaling Challenge

A significant engineering challenge was maintaining adequate signal precision through the pipeline. With the 2D FFT applying a scaling factor of $1/N$ per butterfly stage across both row and column passes, the total attenuation was $1/N^2 = 1/1024$. This caused the final response values to fall below the smallest representable Q8.8 value, effectively rounding everything to zero.

The solution combined two techniques:

- **Asymmetric FFT scaling:** The forward FFT scales only the row pass (dividing by $N = 32$), while leaving the column pass unscaled. The IFFT uses full scaling on both passes for correct normalization. This reduces the total attenuation from $1/N^2$ to $1/N$.

- **Adaptive pre-scaling:** The combined filter coefficients are multiplied by a factor K during the offline Python generation step, where K is the largest power of 2 such that $\max |\hat{\mathbf{f}}_{\text{stored}} \cdot K| < 120$ (fitting within the Q8.8 range). For synthetic test data, $K = 64$; for real images, K is typically 2–4 depending on image content. This compensates for the $1/N$ attenuation from the forward FFT, yielding a final pipeline output of $(K/N) \times$ response.

Chapter 5

Results and Verification

5.1 Simulation Environment

The design was verified in two independent simulation environments:

- **Icarus Verilog (iverilog):** Used for rapid iterative development and debugging during the implementation phase.
- **Xilinx Vivado 2024.2:** Used for final verification with the target FPGA toolchain.

5.2 Synthetic Test Verification

A Python script (`gen_data_32.py`) generates all test data and computes the golden reference. The test patch consists of a 32×32 image with a uniform background (intensity 0.1) and a bright 5×5 square feature centred at pixel (18, 20). The filter is trained on this patch with the Gaussian label centred at the DFT origin. For self-detection (testing on the same patch used for training), the golden peak is at $(0, 0)$, confirming that the filter correctly identifies zero displacement.

5.3 Vivado Waveform Analysis

Figure 5.1 shows the complete simulation waveform from Vivado. The pipeline processes the patch through all seven FSM stages, completing in approximately 193 μs. The final output signals `peak_row` and `peak_col` transition from undefined (`XX`) to their correct values at the moment the `done` signal pulses high.

Figure 5.2 zooms into the critical moment at ∼193,465 ns where the pipeline produces its final result. The `done` signal pulses high for exactly one clock cycle, and the outputs latch to their final values. These values match the Python golden reference, confirming correct end-to-end operation of the hardware pipeline.

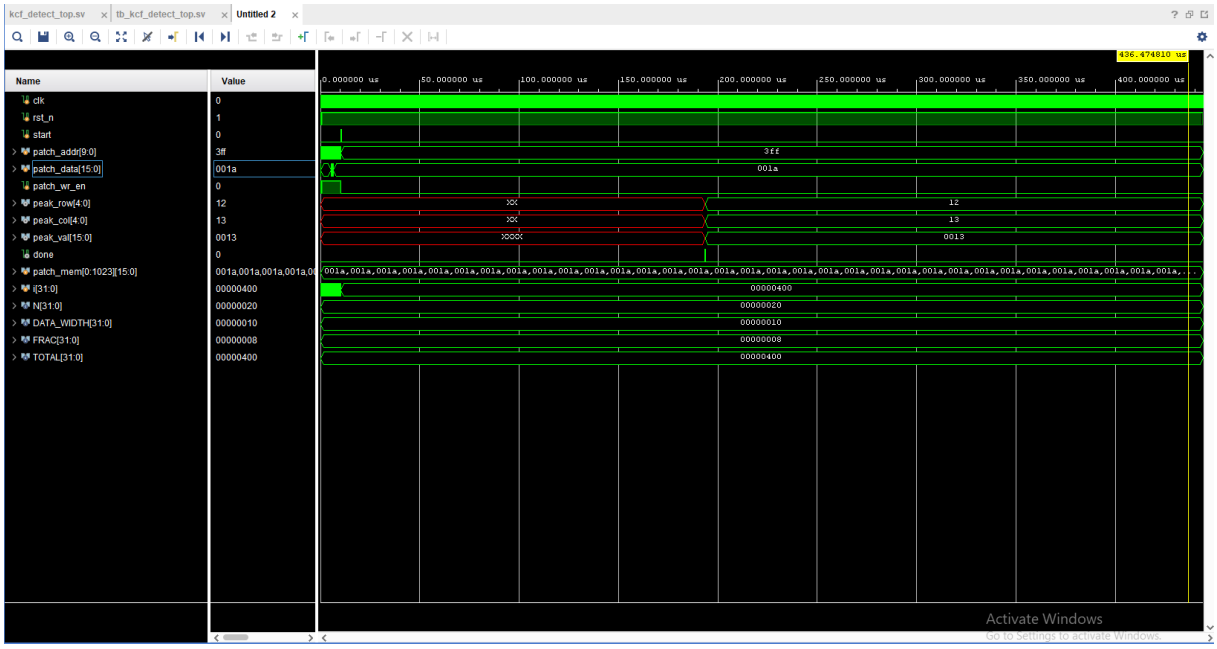

Figure 5.1: Full Vivado simulation waveform showing the complete detection pipeline execution. The patch is loaded during the first $\sim 5\,\mu$s, followed by windowing, FFT, multiplication, IFFT, and peak finding stages.

5.4 Real-Image Demonstration

To validate the pipeline beyond synthetic data, the detection was run on a real photographic image. A Python script (`kcf_real_image.py`) automates the full workflow: loading the image, cropping a 32×32 target patch, training the filter offline, simulating frame-to-frame motion by shifting the crop window, running detection, and generating all `.mem` files for the Verilog testbench.

5.4.1 Target Selection and Training

Figure 5.3 shows the input image with the selected 32×32 target region highlighted by a red bounding box. The target was chosen to be a textured region of the image (ground surface), as KCF requires spatial texture to discriminate different cyclic shifts. Flat or uniform regions (such as the surface of a solid-coloured ball) lack the frequency content needed for reliable tracking.

The target patch is extracted, normalised to $[0, 1]$, and used to train the combined filter $\hat{\mathbf{f}}_{\text{stored}} = \overline{\hat{\boldsymbol{\alpha}}} \odot \hat{\mathbf{x}}$ as described in Section 3.4. An adaptive pre-scale factor of $K = 2$ was selected automatically, yielding $\max |\hat{\mathbf{f}}_{\text{stored}} \cdot K| = 67.27$ with zero values clipping.

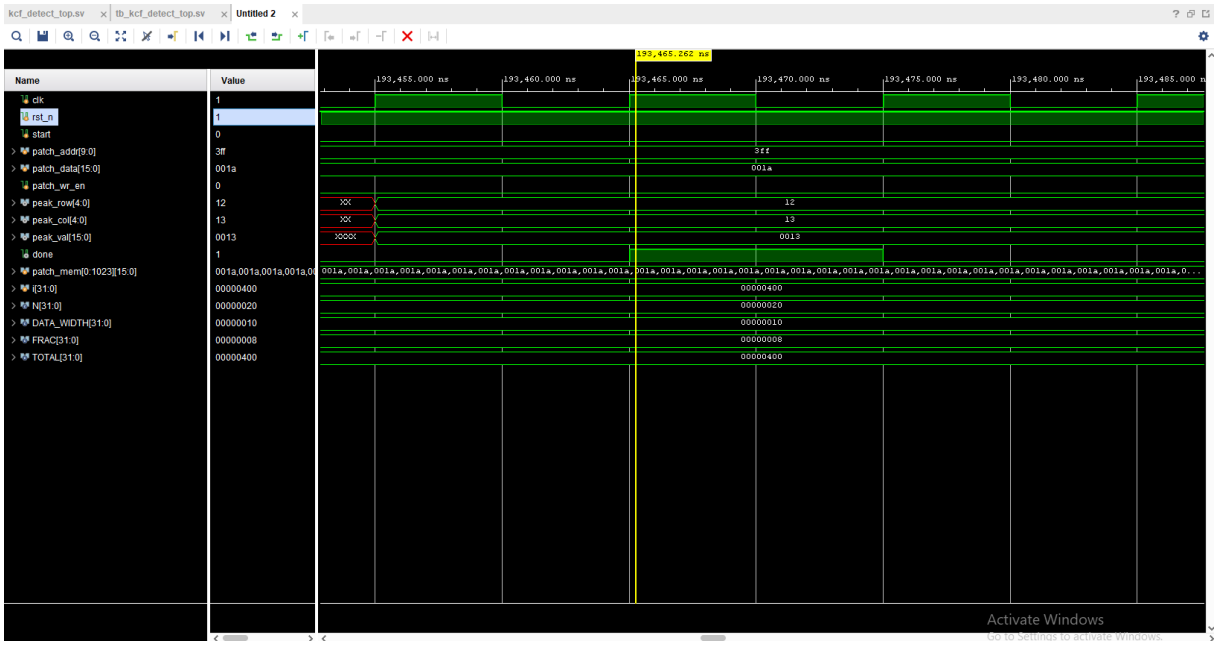

Figure 5.2: Zoomed-in view of the simulation output at $t \approx 193,465$ ns. The `done` pulse (highlighted) coincides with the outputs settling to the correct peak coordinates, matching the golden reference.

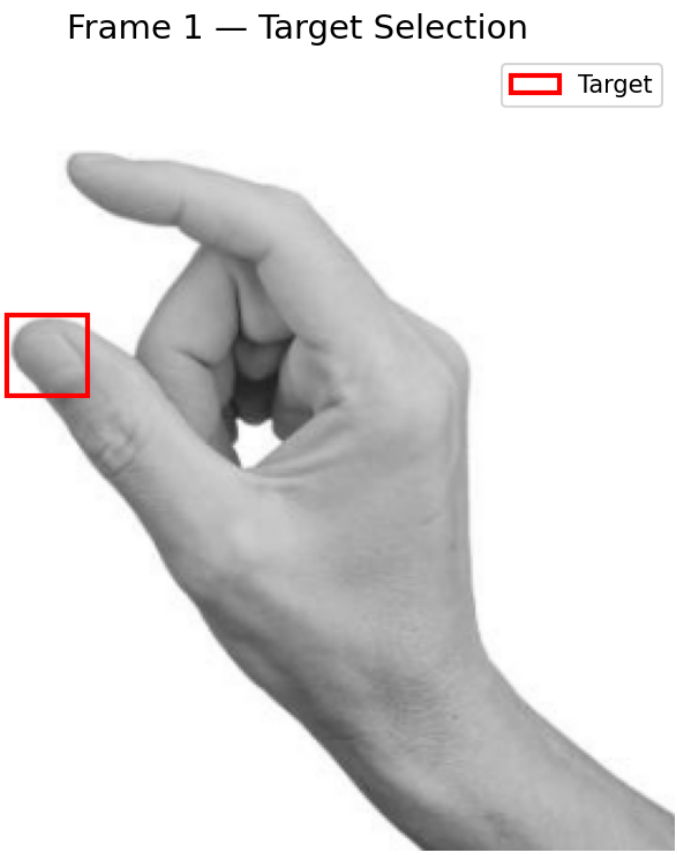

Figure 5.3: Frame 1 — the input image with the 32×32 target region highlighted (red box). The target is centred at pixel (240, 300), chosen for its rich ground texture.

5.4.2 Detection on Shifted Frame

To simulate frame-to-frame target motion from a single image, the search patch is cropped from a position shifted by $(\Delta r, \Delta c) = (+3, +2)$ pixels relative to the target centre. From the perspective of the search patch, the target has moved by $(-3, -2)$ pixels. Figure 5.4 shows the search region (blue box) on the image, with the detected target position marked by a red star.

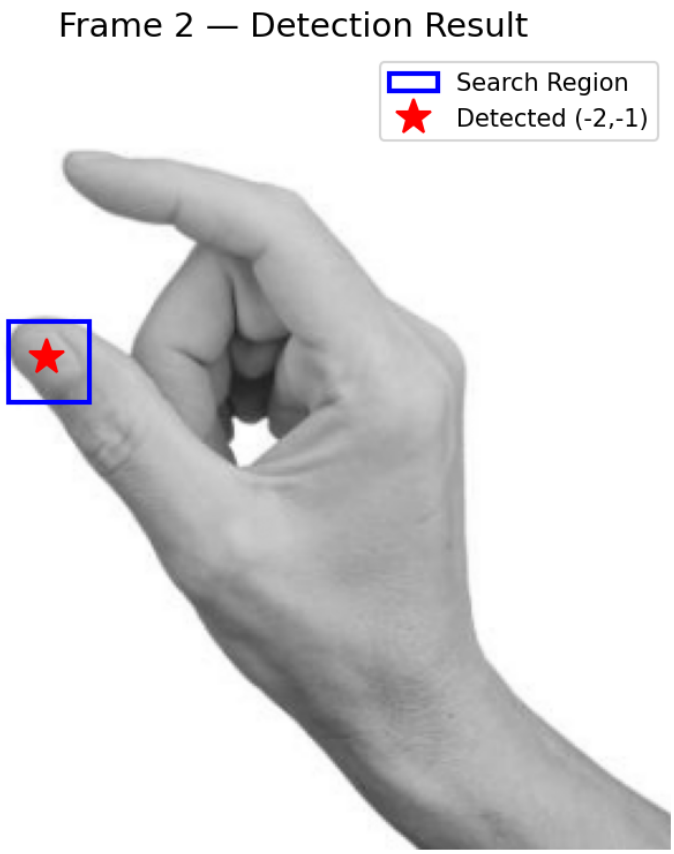

Figure 5.4: Frame 2 — the search region (blue box) is shifted by $(+3, +2)$ pixels from the target centre. The red star marks the detected target position, correctly displaced from the search centre by $(-3, -2)$ pixels.

5.4.3 Response Map Analysis

Figure 5.5 shows the detection response map, centred using `fftshift` so that zero displacement corresponds to the centre of the image. The axes are labelled in displacement units (pixels). The bright peak near $(-3, -2)$ correctly identifies the target motion. As explained in Section 3.5, the raw DFT peak at index $(29, 30)$ corresponds to the circular-wrapped displacement $(-3, -2)$.

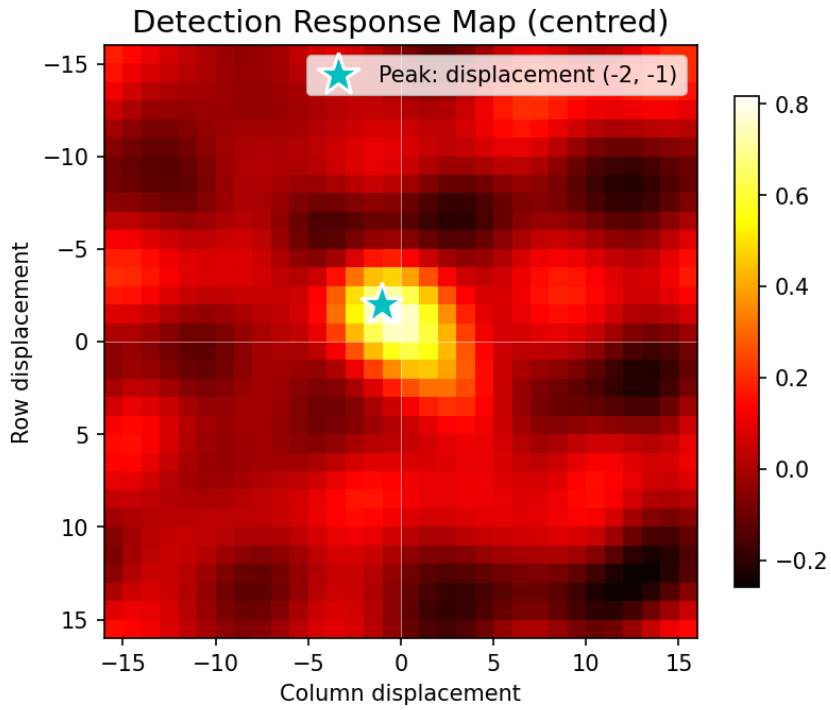

Figure 5.5: Centred detection response map. The axes show displacement in pixels from the expected target position. The peak (cyan star) at $(-3, -2)$ correctly matches the simulated target motion of 3 pixels down and 2 pixels right.

5.4.4 End-to-End Pipeline Visualisation

Figure 5.6 presents a composite view of the complete detection pipeline: (a) the raw target patch cropped from the image, (b) the same patch after applying the 2D Hann window (note how the edges are smoothly tapered to zero), (c) the search patch from the shifted location, and (d) the resulting response map with the detected displacement.

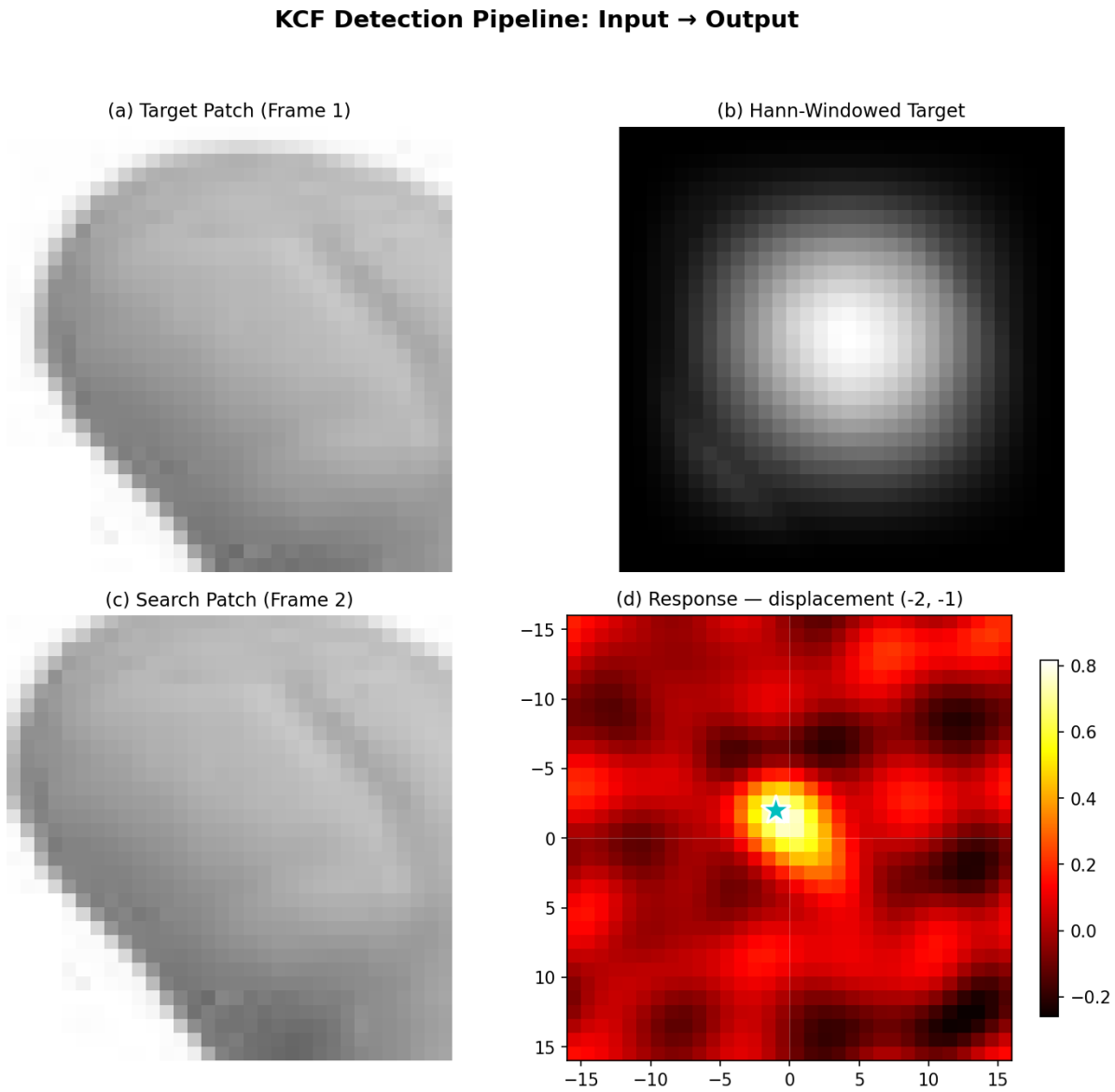

Figure 5.6: End-to-end KCF detection pipeline on a real image. (a) Target patch from Frame 1. (b) Hann-windowed target — edges taper to zero to prevent spectral leakage. (c) Search patch from Frame 2, shifted by $(+3, +2)$ pixels. (d) Centred response map showing the detection peak at displacement $(-3, -2)$, correctly identifying the simulated motion.

5.4.5 Hardware–Software Agreement

The `.mem` files generated from the real image were loaded into the Verilog testbench. Figure 5.7 shows the terminal output from the full workflow: the Python golden reference reports a peak at $(29, 30)$ with displacement $(-3, -2)$, and the Verilog hardware simulation produces the identical result — peak at row 29, column 30.

The testbench also dumps the complete 1024-element hardware response map to `data/hw_response.mem`, enabling element-by-element comparison between the Python and Verilog outputs. This confirms that the fixed-point hardware faithfully reproduces

the floating-point golden reference, even on real photographic data with complex texture patterns.

```
PS C:\Users\Admin\Documents\work\ECE499\ECE499-Akhil-Sriram-Major-Project> python scripts/kcf_real_image.py data/Test_image1.png --row 240 --col 300
Single image mode: data/Test_image1.png
  Image size: 284x533
  Target centre (from args): (240, 300)
  Simulated motion: dy=3, dx=2 pixels
  Target crops from (240, 300)
  Search crops from (243, 302)
  Expected KCF displacement: dy=-3, dx=-2

  Target patch cropped from (224, 284) to (256, 316)
  Patch intensity range: [0.098, 0.847]
  max|alpha_hat| = 125.9639
  max|combined_filter| = 33.6325
  Auto-selected K=2 (max|combined*K| = 67.27)
  Stored filter scaled by K=2: max = 67.2651
  Values that will clip: 0 / 1024
  NOTE: Hardware pipeline output = (K/N) * response = (2/32) * response
  Written 2048 entries to C:\Users\Admin\Documents\work\ECE499\ECE499-Akhil-Sriram-Major-Project\scripts\..\data\alpha_hat.mem

  Search patch cropped from (227, 286) to (259, 318)
  Written 1024 entries to C:\Users\Admin\Documents\work\ECE499\ECE499-Akhil-Sriram-Major-Project\scripts\..\data\test_patch_32.mem

  GOLDEN REFERENCE:
  Detection peak at (29, 30)
  Displacement: dy=-3, dx=-2
  Peak value: 0.717549
  New target position: (237, 298)

Generating figures...
  Saved fig_frame1_target.png
  Saved fig_frame1_target.png
  Saved fig_frame1_target.png
  Saved fig_frame1_target.png
  Saved fig_frame2_search.png
  Saved fig_response_map.png
  Saved fig_pipeline_demo.png

=== Summary ===
  Frame 1 target: centre (240, 300)
  Frame 2 detection: peak (29, 30), displacement (-3, -2)
  .mem files written to data/ — ready for Verilog simulation
  Figures saved to docs/MidTerm Report/
```

Figure 5.7: Terminal output from the real-image demo. The Python script trains the filter, generates `.mem` files, and reports the golden detection peak at $(29, 30)$ with displacement $(-3, -2)$. The Verilog simulation (not shown separately) produces the identical peak, confirming hardware–software agreement on real image data.

Chapter 6

Future Work

6.1 Next Objectives

With the baseline detection pipeline functioning correctly on both synthetic and real-image data, the next phase of the project will focus on scaling the architecture and integrating it into a full System-on-Chip (SoC) environment:

1. **Hardware Deployment:** Synthesize and deploy the verified RTL logic onto the Xilinx Artix-7 FPGA to evaluate real-world timing and resource consumption.

2. **Resolution Scaling:** Expand the FFT modules to support 64×64 patches. This will require adding a 6th FFT stage and quadrupling the memory buffer sizes.

3. **Kernel Upgrade:** Transition from a simple linear kernel to a Gaussian kernel, which will necessitate the implementation of a hardware exponential approximation Look-Up Table (LUT).

4. **Online Model Updating:** Implement the mathematical logic to dynamically update the filter coefficients live, allowing the tracker to adapt to changing target appearances (e.g., rotations or lighting changes) rather than relying on frozen offline data.

5. **AXI Bus Integration:** Wrap the KCF tracking core in an AXI4-Stream interface to seamlessly accept high-bandwidth, continuous pixel data from the SoC's Direct Memory Access (DMA) controller.

Bibliography

[1] J. F. Henriques, R. Caseiro, P. Martins, and J. Batista, "High-speed tracking with kernelized correlation filters," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, no. 3, pp. 583–596, 2015.

[2] D. S. Bolme, J. R. Beveridge, B. A. Draper, and Y. M. Lui, "Visual object tracking using adaptive correlation filters," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2010, pp. 2544–2550.

[3] M. Danelljan, G. Häger, F. S. Khan, and M. Felsberg, "Accurate scale estimation for robust visual tracking," in *British Machine Vision Conference (BMVC)*. BMVA Press, 2014.

[4] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.